# PROJECT REPORT

# RESEARCH ASSISTANT

## NLP-Based Research Paper Analysis, Discovery, Comparison and Recommendation System

| | |
|---|---|
| **Student Name** | Mailapilli Deepthi |
| **Roll Number** | 323103282055 |
| **Department & Section** | 4 CSM - 1 |
| **Subject** | Natural Language Processing |
| **Academic Year** | 2026-2027 |

Submitted in partial fulfillment of the academic requirements

# ABSTRACT

Research Assistant is a browser-based application for reading, discovering, comparing and organizing research papers. It combines a single-page HTML, CSS and JavaScript interface with a Python FastAPI backend. The application is designed to run locally without model downloads or AI API keys; an internet connection is needed for scholarly search and selected chat lookups.

The Analyze Paper module extracts text from uploaded PDFs and produces an overall and section-wise summary, key paper information, structured extraction and research-gap suggestions. The NLP pipeline uses pypdf, sentence segmentation, TF-IDF features, sentence-similarity scoring and rule-based patterns. Find Papers searches Crossref metadata for selected publisher DOI prefixes, OpenAlex and arXiv, then ranks results by availability of an abstract and free PDF. Recommendations use keywords from the analyzed paper.

Additional modules compare two or three PDFs, draft a related-work paragraph with APA references, answer questions about the analyzed paper and research concepts, and save papers or analyses with notes in browser localStorage. Analysis, comparison and literature-review outputs can be downloaded as PDF or Word reports.

The system is intended as an academic reading aid. Its summaries and extracted facts are heuristic outputs and should be checked against the source paper. It does not replace a researcher’s critical review or verify the scientific correctness of a paper.

**Keywords:** Natural Language Processing, research papers, PDF analysis, TF-IDF, information extraction, literature review

# TABLE OF CONTENTS


- 1. Introduction
- 2. Problem Statement and Motivation
- 3. Aim, Objectives and Scope
- 4. Existing System and Proposed System
- 5. Tools and Technologies Used
- 6. System Architecture and Workflow
- 7. System Modules
- 8. NLP Methods and Data Sources
- 9. Implementation Details
- 10. Screenshot Places
- 11. Testing and Evaluation
- 12. Results and Discussion
- 13. Limitations and Responsible Use
- 14. Future Enhancements
- 15. Conclusion
- References
- Appendix A: Example User Tasks
- Appendix B: Setup and Operation Checklist

# 1. INTRODUCTION

Research papers are a primary source of technical and scientific knowledge, but a reader may need to search several databases, scan long PDFs, identify methods and datasets, compare findings, and keep notes across many papers. These activities can be time-consuming, especially when beginning a literature review.

Research Assistant brings several of these activities into one local web application. A user can upload a paper for a structured analysis, search scholarly sources by topic and year, ask questions about the analyzed paper, and compare multiple papers. Saved items and generated reports support later review.

## 1.1 Need for the Project

- Reduce the effort required to locate key information in a research PDF.
- Support topic-based discovery from multiple scholarly sources.
- Make comparison of methods, datasets, metrics, results and limitations more direct.
- Demonstrate practical NLP techniques in an end-to-end software project.
- Keep useful papers and analysis outputs organized for future reference.

## 1.2 Project Overview

The project follows a browser-client and API-server design. The browser renders the interface and sends requests to FastAPI. Python modules perform PDF processing, NLP, scholarly search, comparison, chat response construction and report generation. The browser stores the personal library locally; the backend does not use a project database.

# 2. PROBLEM STATEMENT AND MOTIVATION

## 2.1 Problem Statement

Students and researchers often need to discover papers, understand their contents, compare evidence across studies and retain notes. These tasks are commonly split across search websites, PDF readers, spreadsheets and document editors. Important details may be missed when papers are long or use inconsistent section layouts.

The project addresses this workflow problem by providing one interface for paper analysis, search, comparison, question answering, recommendations, library storage and report export. The software does not claim to validate research quality or replace the original sources.

## 2.2 Motivation

- A compact paper summary can help a reader decide which sections to inspect more closely.
- Structured extraction makes selected paper facts easier to compare.
- Search across several indexes can expose papers that a single-source search may miss.
- A local library can retain saved items and personal notes in the user's browser.
- A modular implementation provides a practical demonstration of classic NLP and web API integration.

# 3. AIM, OBJECTIVES AND SCOPE

## 3.1 Aim

To develop a local web-based research assistant that applies NLP techniques to help users analyze, discover, compare and organize research papers.

## 3.2 Objectives

- Extract readable text from uploaded PDF research papers.
- Generate overall and section-wise extractive summaries.
- Identify selected authors, dates, keywords, methods, datasets, metrics and named entities.
- Highlight possible limitations and suggestions as heuristic research-gap prompts.
- Search Crossref, OpenAlex and arXiv using a topic and publication-year range.
- Compare two or three PDF papers and draft a related-work paragraph.
- Provide question answering about the analyzed paper, NLP concepts and related topics.
- Save papers and analyses with notes in browser storage and export reports in PDF or Word.

## 3.3 Scope

The current implementation supports text-based PDF extraction, metadata search from configured providers, rule-based and TF-IDF-based NLP, local browser storage, and PDF/DOCX report generation. Search and selected general-topic chat lookups require internet access. Scanned PDFs without an embedded text layer are not processed with OCR.

# 4. EXISTING SYSTEM AND PROPOSED SYSTEM

## 4.1 Existing Workflow

A typical literature-review workflow uses separate tools for finding papers, reading PDFs, making notes, comparing methods and preparing a report. A user may need to manually copy paper details and move between provider websites.

## 4.2 Proposed System

Research Assistant integrates a set of these tasks in a single browser interface. It combines local PDF analysis with multi-source search, paper comparison, research chat, a browser-based library and downloadable reports.

| Aspect | Separate-tool workflow | Research Assistant |
|---|---|---|
| Paper analysis | Read and summarize manually | Summary, key information, extraction and gap prompts |
| Paper discovery | Search provider websites separately | One topic search across selected sources |
| Cross-paper review | Create a comparison manually | Compare 2-3 PDFs in a structured table |
| Paper questions | Search the PDF manually | Chat retrieves relevant text from the current analysis |
| Organization | Maintain separate notes/files | Browser-local saved papers, analyses and notes |
| Output | Prepare a report manually | Export available analysis/review reports as PDF or Word |

*This is a reading and organization aid. Users should verify extracted details and conclusions against the original papers.*

# 5. TOOLS AND TECHNOLOGIES USED

| Technology | Role in the project |
|---|---|
| Python | Backend implementation and NLP processing. |
| FastAPI | HTTP API routes, request parsing and validation. |
| Uvicorn | Runs the local ASGI web application. |
| HTML, CSS and JavaScript | Single-page user interface and browser interactions. |
| scikit-learn | TF-IDF vectorization and sparse similarity calculations. |
| NumPy | Numerical operations used in text scoring. |
| pypdf | Extracts text from uploaded PDF documents. |
| ReportLab | Creates PDF report downloads. |
| python-docx | Creates editable Word report downloads. |
| Requests | Calls public scholarly APIs and selected web sources. |
| Crossref, OpenAlex, arXiv | External metadata and scholarly search sources. |
| Browser localStorage | Stores the user's saved library and notes locally. |

## 5.1 Runtime Requirements

Install the packages listed in requirements.txt, then run python run.py from the project directory. The default launcher starts the app at http://127.0.0.1:8000 and opens the browser. Internet connectivity is needed for live paper search and some chat sources; local analysis and report generation run on the machine.

# 6. SYSTEM ARCHITECTURE AND WORKFLOW

| Layer | Components | Responsibility |
|---|---|---|
| Presentation | Browser page, HTML/CSS/JavaScript | Collects user actions, displays results and saves local library data. |
| API | FastAPI application | Validates requests and routes them to processing modules. |
| NLP and document processing | nlp.py, pypdf, scikit-learn | Extracts PDF text, summarizes, extracts facts and finds relevant sentences. |
| Research services | search.py, compare.py, chat.py | Searches providers, compares paper profiles and prepares answers/reviews. |
| Report output | report.py, ReportLab, python-docx | Builds PDF and Word downloads from analysis/review data. |
| External sources | Crossref, OpenAlex, arXiv, Wikipedia | Provide metadata, scholarly search results or selected topic information. |

## 6.1 Main Workflow

1. The user chooses a module and provides a PDF, search topic, question or saved item.
2. JavaScript sends an HTTP request to the corresponding FastAPI endpoint.
3. The backend validates the request and invokes NLP, search, comparison or report logic.
4. The backend returns JSON data or a downloadable document.
5. The browser renders the result and, when requested, stores selected items locally.

## 6.2 Analysis Data Flow

PDF bytes are read by the upload route and passed to pypdf. Extracted text is split into sections and sentences. The NLP functions produce the summary, key information, extraction and gap-analysis blocks. The API returns these values to the interface and stores the current full analysis in server memory for chat and recommendations.

# 7. SYSTEM MODULES

## 7.1 Analyze Paper

Accepts a PDF up to the configured 200 MB limit. It returns an overall summary, summaries by detected section, key paper information, structured extraction and research-gap prompts. Uploaded scanned documents that yield too little readable text are rejected.

## 7.2 Find Papers

Searches selected sources by topic, year range and result count. Crossref is queried through configured DOI prefixes for IEEE, Springer, Elsevier and ACM. OpenAlex and arXiv are also supported. OpenAlex can enrich records with abstracts and available open-access PDF links. Results are grouped by analyzability and download availability.

## 7.3 Chat

Answers questions using the currently analyzed paper, a built-in research/NLP glossary and selected external sources for general topics. For paper-specific questions, TF-IDF retrieval ranks sentences from the analysis text and formats the retrieved evidence in a response.

## 7.4 Recommendations

Uses the top keywords from the current analysis to formulate a scholarly search. The user can set a date range and result count. The current paper is filtered from the returned results.

## 7.5 Compare Papers and Literature Review

Accepts two or three PDFs and builds paper profiles for a side-by-side table of problem, methods, datasets, metrics, results and limitations. A related-work draft can be generated from compared papers or selected saved analyses, with APA-style references.

## 7.6 Library and Reports

Papers and analyses can be saved with notes in browser localStorage. Saved analyses can be viewed again and selected to draft a literature review. Analysis, comparison and review outputs can be downloaded as PDF or editable DOCX documents.

# 8. NLP METHODS AND DATA SOURCES

| Task | Implemented method | Purpose |
|---|---|---|
| PDF text extraction | pypdf page-text extraction | Read text embedded in a PDF without an OCR dependency. |
| Section detection | Regular-expression heading patterns | Divide documents into recognized sections where possible. |
| Sentence processing | Whitespace normalization and punctuation-based segmentation | Prepare candidate sentences for summaries and retrieval. |
| Summary | TF-IDF sentence vectors; sum of sentence similarities | Select central sentences for extractive summaries. |
| Keyword extraction | TF-IDF over unigrams and bigrams | Identify salient terms and phrases. |
| Information extraction | Regular expressions and curated vocabularies | Find selected entities, methods, datasets, metrics and dates. |
| Research-gap prompts | Rule checks for missing evaluation/reporting evidence | Suggest limitations to inspect; not a definitive quality judgment. |
| Question retrieval | TF-IDF sentence matching and cosine similarity | Select text relevant to a question about the current paper. |
| Paper comparison | Extracted profiles and rule-based grouping | Present common and paper-specific information side by side. |

## 8.1 Data Handling

The application does not include a trained model or a project-managed research corpus. Search results come from public provider APIs, while the NLP vocabulary and glossary are source-code resources. Saved-library records are kept in the current browser profile. The analyzed paper used for server-side chat and recommendations is held in application memory.

*The term "TextRank-style" describes the centrality-inspired sentence scoring used here; the implementation is a lightweight TF-IDF similarity score rather than a full graph-based TextRank algorithm.*

# 9. IMPLEMENTATION DETAILS

## 9.1 Project Structure

```
research-assistant/
  run.py
  requirements.txt
  frontend/
    index.html
  backend/
    app.py
    nlp.py
    search.py
    chat.py
    knowledge.py
    compare.py
    citations.py
    report.py
```

## 9.2 Main API Routes

| Route | Method | Purpose |
|---|---|---|
| / | GET | Serve the single-page application. |
| /api/analyze | POST | Analyze an uploaded PDF. |
| /api/analyze-paper | POST | Analyze a paper from a search result or abstract. |
| /api/find | GET | Search selected scholarly sources. |
| /api/chat | POST | Answer a question using current state and available knowledge. |
| /api/recommend | GET | Find papers using current-analysis keywords. |
| /api/compare | POST | Compare two or three uploaded PDFs. |
| /api/litreview | POST | Generate a related-work draft from paper profiles. |
| /api/report | POST | Return a PDF or DOCX analysis, comparison or review. |
| /api/about | GET | Return project feature and NLP topic information. |

## 9.3 State and Storage

The browser library is stored under a localStorage key in the user's browser. It is not synchronized to a server and may be removed when browser data is cleared. The backend's current analyzed-paper state is held in process memory, so it is reset when the server restarts and is shared by requests to that running app instance.

# 10. SCREENSHOT PLACES

# 11. TESTING AND EVALUATION

Testing should cover the web interface, request validation, NLP behavior and generated downloads. The table records the checks performed while preparing this report; it does not claim a large-scale accuracy or user study.

| Test | Expected result | Observed status |
|---|---|---|
| Open local home page | Research Assistant page is served by FastAPI. | Passed: HTTP 200 on the running local app. |
| Open About information | Feature and NLP topic information is returned. | Passed: HTTP 200; six topic units returned. |
| Request recommendations before analysis | A clear client error asks for a paper analysis first. | Passed: HTTP 400 with the expected message. |
| Compare fewer than two PDFs | Request is rejected with a validation message. | Passed: HTTP 400 for a one-file request. |
| Generate a PDF report | A readable PDF is returned for valid report data. | Passed: HTTP 200; generated PDF parsed successfully. |
| Generate a Word report | An editable DOCX is returned for valid report data. | Passed: HTTP 200; DOCX package integrity verified. |
| Reject a non-PDF upload | The analysis route rejects a file that is not a PDF. | Passed: HTTP 400 with the expected message. |

## 11.1 Evaluation Notes

- A server response confirms route availability, not the semantic accuracy of extracted facts.
- Summary quality depends on PDF text quality, sentence segmentation and the selected sentences.
- Search result quantity and metadata depend on external providers and the query.
- A research-gap prompt means a pattern was not detected; it does not prove that the paper lacks the practice.
- Manual review with representative papers is required before academic submission or research use.

# 12. RESULTS AND DISCUSSION

The implemented application provides an end-to-end workflow that connects paper upload, NLP analysis, scholarly search, comparison, chat, recommendations, local organization and report downloads. It demonstrates how lightweight NLP methods can be combined with a web API and a browser interface without requiring a hosted language-model API.

## 12.1 Functional Outcomes

- Text-based research PDFs can be analyzed into summaries and structured information.
- Search results from multiple configured scholarly sources are presented in grouped order.
- Two or three analyzed papers can be compared using common information categories.
- Related-work drafts use the extracted profiles and include generated APA references.
- Saved records and notes remain in the same browser profile.
- Supported reports are generated as PDF or Word documents.

*Screenshots from the running Research Assistant application are included in Appendix C.*

## 12.2 Discussion

The implementation favors transparency and low setup cost over deep semantic understanding. TF-IDF and pattern rules are lightweight and do not require model downloads, but they are less robust than modern contextual language models. The interface should therefore be treated as a navigation and drafting aid, not an authority on a paper's claims.

# 13. LIMITATIONS AND RESPONSIBLE USE

- No OCR is included; scanned or image-only PDFs may produce no usable text.
- Section recognition, author extraction, entity detection and research-gap checks use heuristics and may be incomplete or incorrect.
- TF-IDF retrieval uses word overlap and may miss paraphrases or conceptual relationships.
- Search requires external services; provider limits, outages and incomplete metadata can affect results.
- The chat and recommendation context for the analyzed paper is stored in server memory and is lost on restart.
- Library content is stored in browser localStorage and is not encrypted, backed up or shared across devices.
- The app has no user authentication or multi-user access control; it is intended for local use, not direct public deployment.
- Generated summaries and related-work text can omit nuance. Verify every statement with the original PDF.
- The current chat does not provide verified PDF page-number citations for each answer.

## 13.1 Responsible Use

Do not submit automatically generated analysis as a substitute for reading the source paper. Check author names, dates, methods and results against the PDF or publisher record. Respect copyright and access terms when downloading papers. Do not expose the local development server to an untrusted network without adding appropriate security controls.

# 14. FUTURE ENHANCEMENTS

- Add OCR support for scanned PDFs and robust support for multi-column layouts.
- Provide page-level evidence links for extracted facts and chat answers.
- Add an optional semantic-embedding retrieval path while retaining a no-key local mode.
- Improve section and entity extraction with configurable evaluation datasets.
- Add multi-paper topic modeling, clustering and visual trend analysis.
- Support export of saved library data and secure backup/restore.
- Add user-selectable language and accessibility improvements.
- Add automated regression tests and a representative evaluation corpus.
- Provide authentication and deployment hardening before any shared or public hosting.

# 15. CONCLUSION

Research Assistant demonstrates a practical application of Natural Language Processing for academic paper workflows. It brings PDF text analysis, multi-source search, recommendations, paper comparison, literature-review drafting, chat, browser-local organization and report export into a single local web application.

The system uses pypdf, TF-IDF, cosine similarity, regular-expression patterns, FastAPI and browser technologies. This design keeps the project lightweight and avoids API keys for core paper analysis. Its main trade-off is that heuristic NLP may produce incomplete or incorrect results.

The project is most useful as a starting point for reading and organizing papers. Its outputs should always be checked against the original scholarly sources, and the future enhancements identified in this report can improve evidence traceability, robustness and usability.

# REFERENCES

6. FastAPI. (n.d.). FastAPI documentation. https://fastapi.tiangolo.com/
7. scikit-learn developers. (n.d.). Feature extraction and TF-IDF documentation. https://scikit-learn.org/
8. pypdf contributors. (n.d.). pypdf documentation. https://pypdf.readthedocs.io/
9. Crossref. (n.d.). REST API documentation. https://www.crossref.org/documentation/retrieve-metadata/rest-api/
10. OpenAlex. (n.d.). API documentation. https://docs.openalex.org/
11. arXiv. (n.d.). Export API user manual. https://info.arxiv.org/help/api/user-manual.html
12. ReportLab. (n.d.). ReportLab documentation. https://docs.reportlab.com/
13. python-docx. (n.d.). python-docx documentation. https://python-docx.readthedocs.io/

*The project's implementation files and README were also used to prepare this report. Web documentation links were accessed on 9 November 2026.*

# APPENDIX A: EXAMPLE USER TASKS

| User task | Expected application behavior |
|---|---|
| Analyze a PDF | Display summary, section-wise summary, key information, extraction and research-gap prompts. |
| Find recent papers | Search selected providers for the entered topic within the chosen year range. |
| Ask about the active paper | Retrieve relevant sentences from the analyzed paper and prepare a response. |
| Get recommendations | Search using the active paper's extracted keywords. |
| Compare papers | Display a side-by-side comparison for two or three PDF files. |
| Draft related work | Generate a draft paragraph and APA references from selected paper profiles. |
| Save and revisit | Store selected papers/analyses and notes in browser localStorage. |
| Download a report | Return the supported analysis, comparison or review as PDF or DOCX. |

# APPENDIX B: SETUP AND OPERATION CHECKLIST

14. Install a supported Python version and open a terminal in the project directory.
15. Install dependencies with: python -m pip install -r requirements.txt
16. Start the application with: python run.py
17. Open http://127.0.0.1:8000 if the browser does not open automatically.
18. Upload a text-based PDF or use Find Papers to begin.
19. Use Compare Papers to select two or three PDFs.
20. Save useful papers or analyses in Library; remember that this data is browser-local.
21. Use PDF report or Word report actions to download supported outputs.
22. Stop the local server with Ctrl+C in the terminal when finished.

*For a final submission, insert the actual screenshots in Section 10, verify the student and institution details, and review the report formatting in Word/PDF.*

# APPENDIX C: APPLICATION SCREENSHOTS

Screenshots captured from the running Research Assistant application.

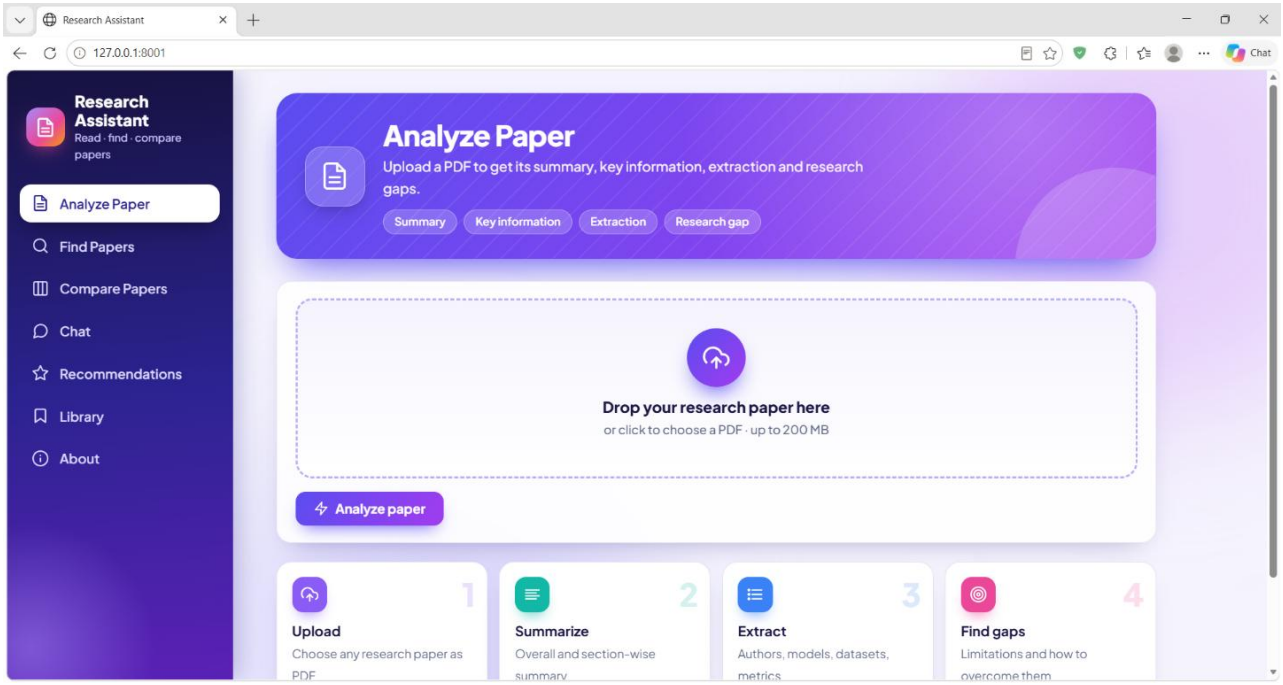



Figure 1. Analyze Paper page before upload

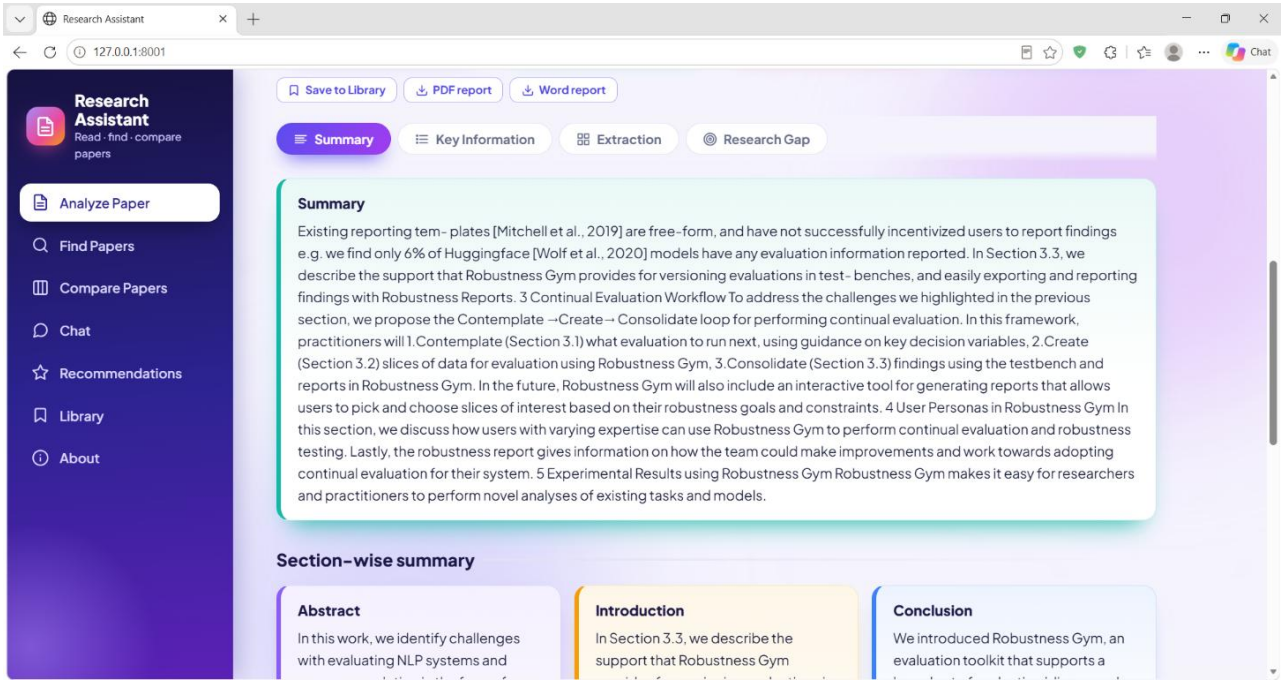



Figure 2. Paper analysis summary and section-wise results

# APPENDIX C: APPLICATION SCREENSHOTS (CONTINUED)

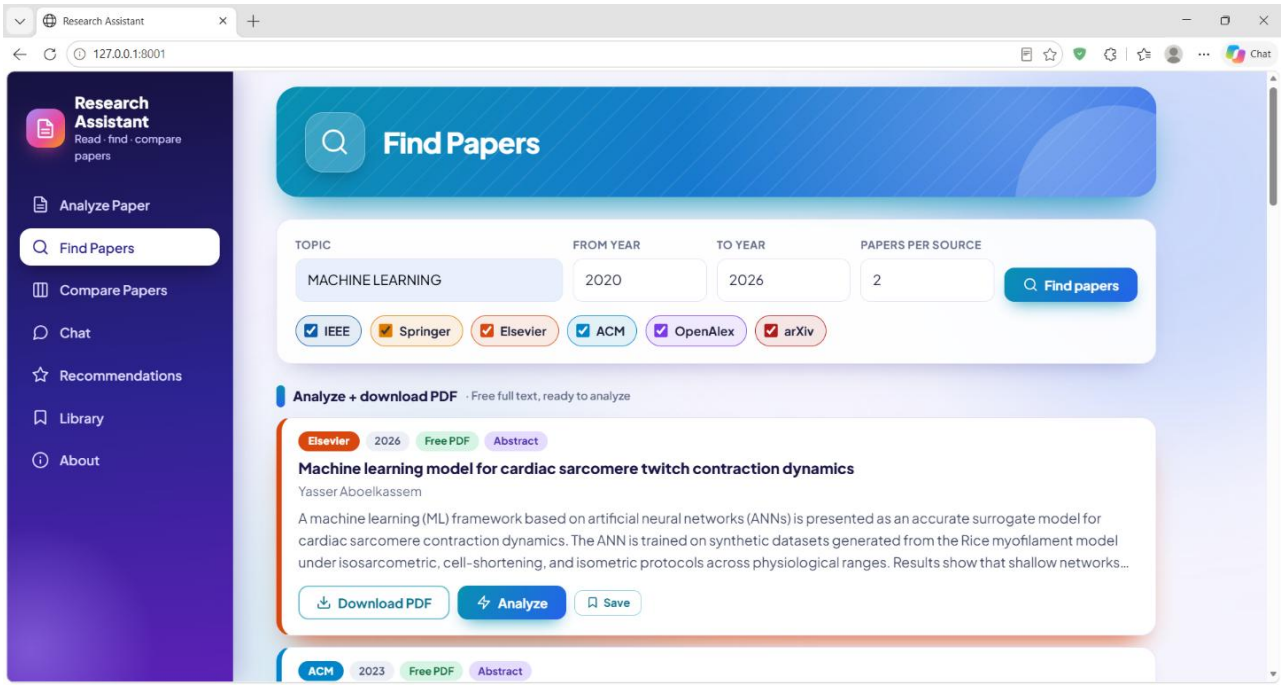



Figure 3. Find Papers search and results

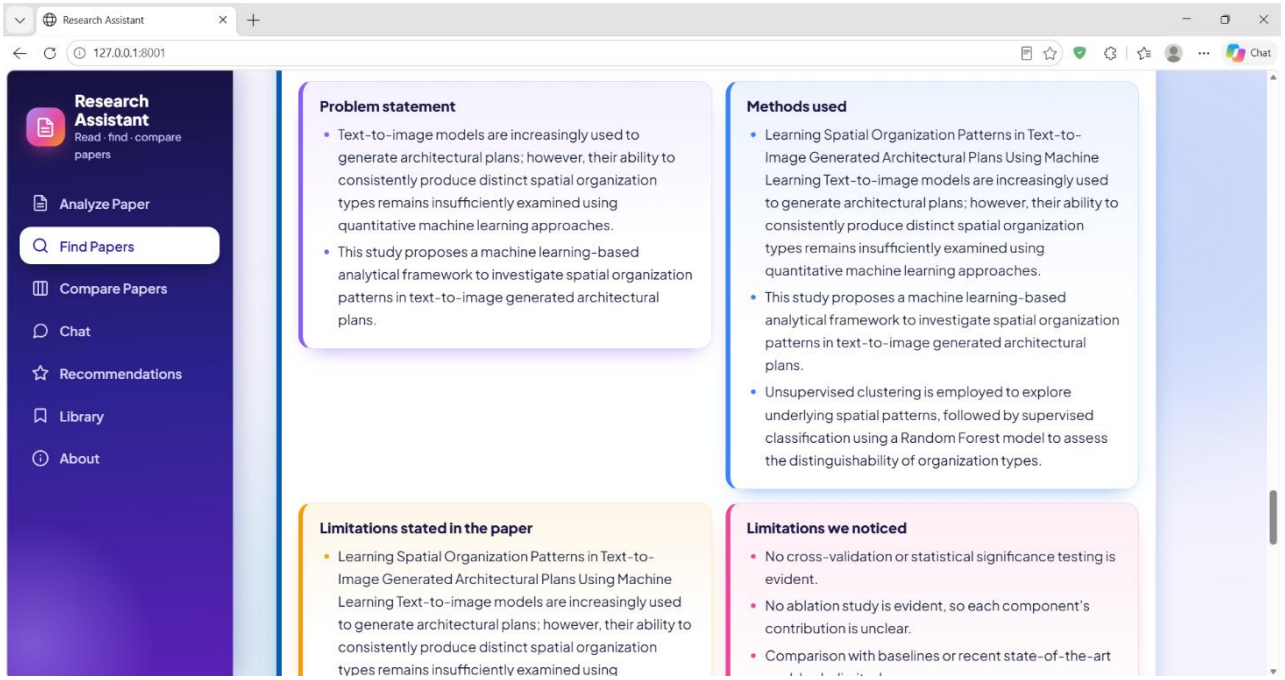



Figure 4. Extracted problem, methods and research-gap analysis

# APPENDIX C: APPLICATION SCREENSHOTS (CONTINUED)

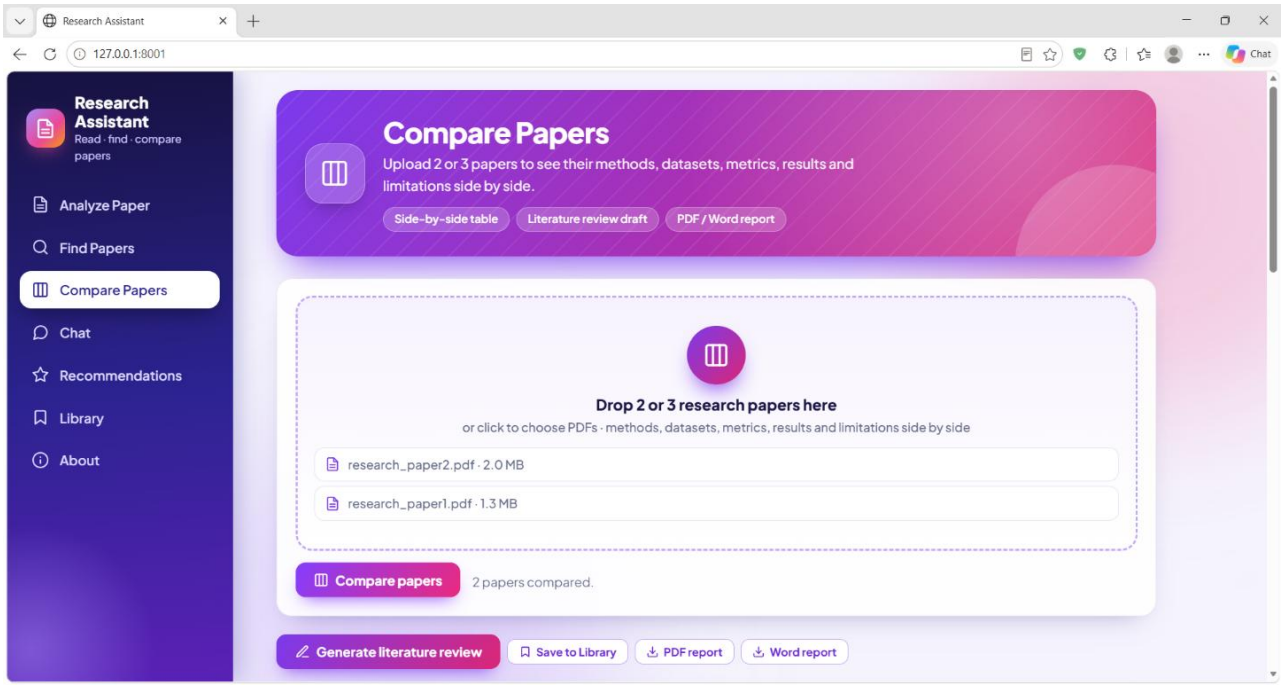



Figure 5. Compare Papers PDF selection and action

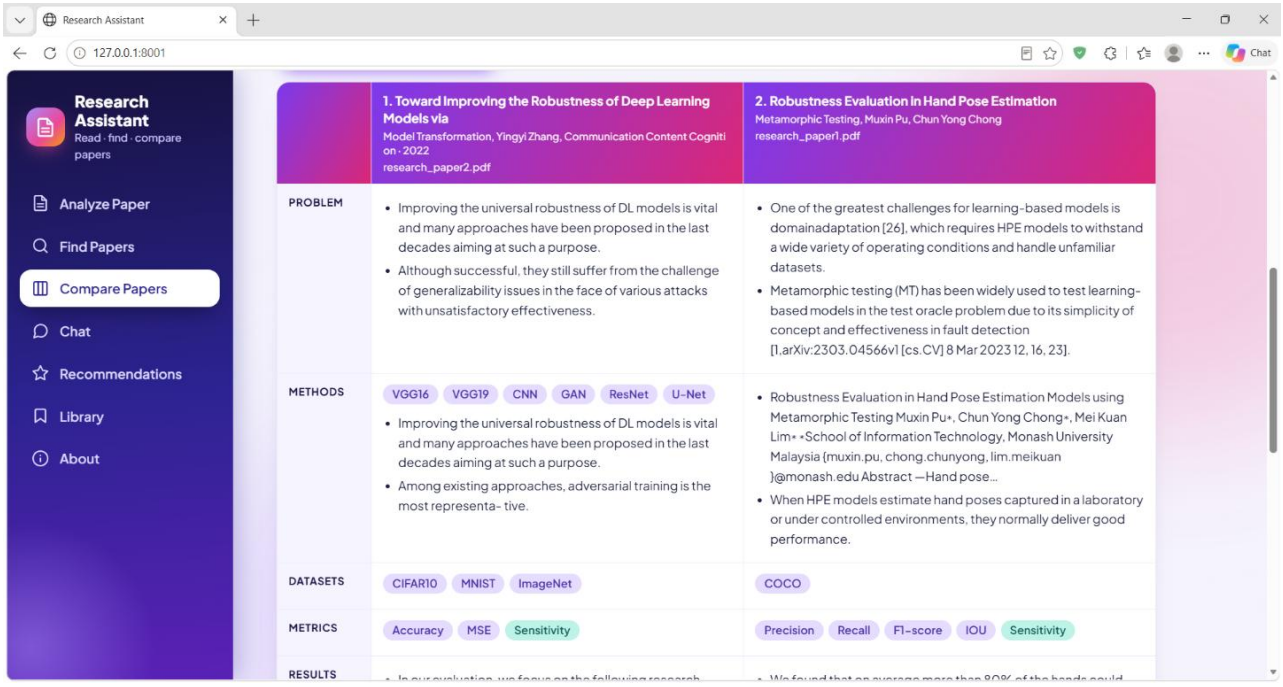



Figure 6. Side-by-side paper comparison table

# APPENDIX C: APPLICATION SCREENSHOTS (CONTINUED)

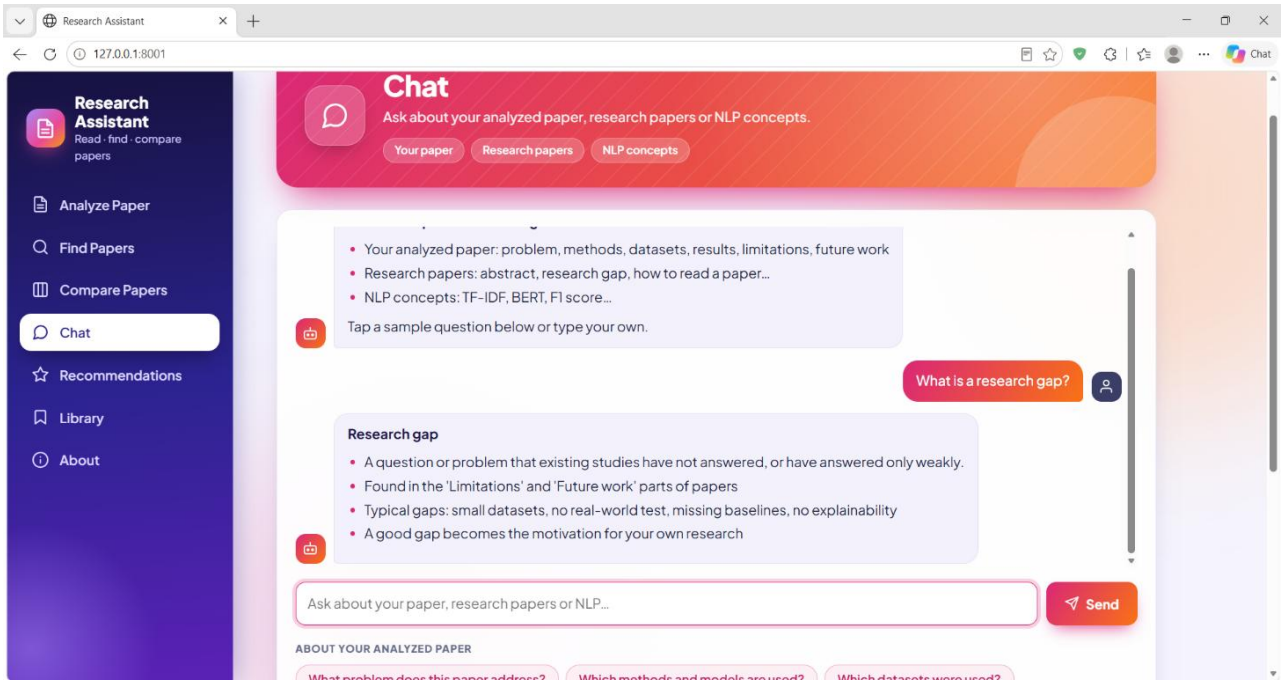



Figure 7. Chat answering a research-gap question

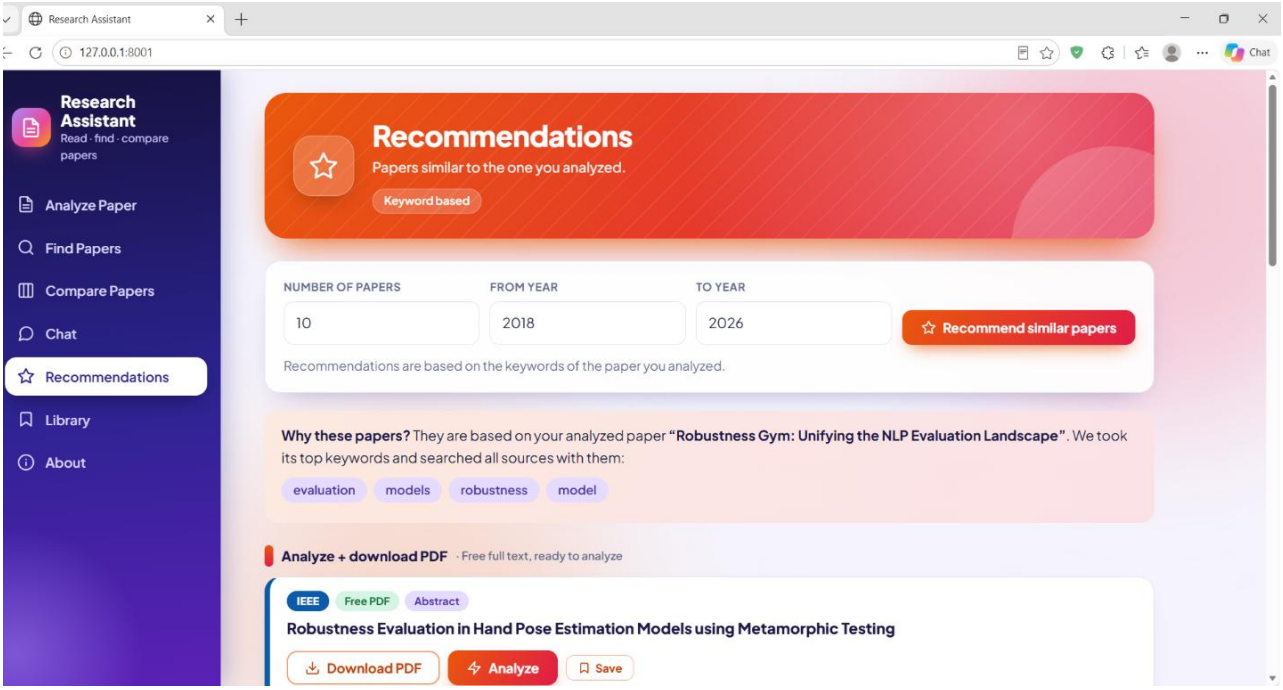



Figure 8. Keyword-based paper recommendations

# APPENDIX C: APPLICATION SCREENSHOTS (CONTINUED)

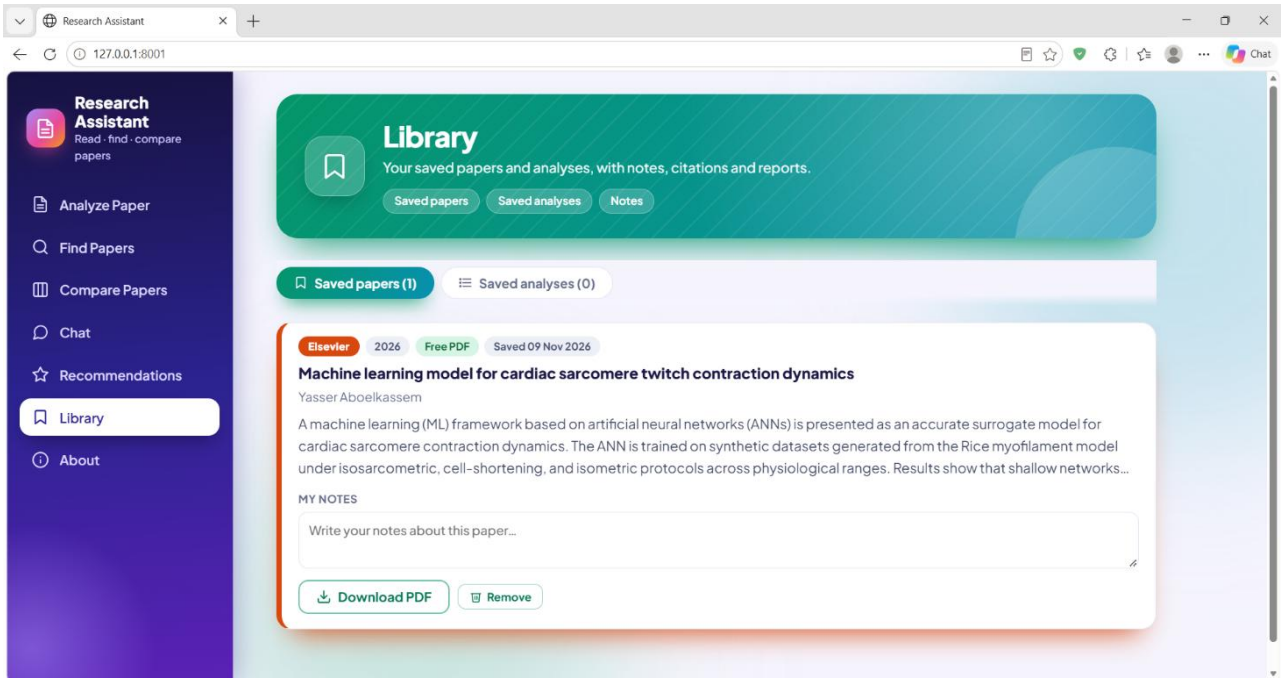



**Figure 9. Saved paper and notes in the Library**